

Tarea 2: 2028

Instituto Tecnológico de Costa Rica
Paradigmas de programación

I Semestre 2026

Estudiante:

Yerik Jaslin Castro
Joseph Solorzano Jackson
Meibel Mora Brenes

Profesor:

Marco Rivera Meneses

Grupo:

01

Fecha:

9 de Abril

1. Descripción General del Sistema

El presente proyecto consiste en la implementación de una versión multijugador distribuida del clásico videojuego *Space Invaders*, desarrollada mediante una arquitectura cliente-servidor. El sistema permite la conexión simultánea de múltiples clientes a un servidor central encargado de administrar el estado global de la partida, incluyendo jugadores, enemigos, bunkers, puntajes, vidas y eventos especiales.

La solución fue desarrollada utilizando distintos lenguajes y tecnologías con el propósito de aplicar conceptos avanzados de programación orientada a objetos, programación concurrente, comunicación por sockets y desarrollo de sistemas distribuidos.

El servidor fue implementado en Java utilizando sockets TCP para permitir la comunicación entre los distintos clientes conectados. Por otra parte, el cliente jugador fue desarrollado en lenguaje C utilizando la biblioteca gráfica Raylib para la representación visual de los elementos del juego.

Adicionalmente, el proyecto contempla la integración de una Raspberry Pi Pico como dispositivo de control físico externo, así como la implementación de clientes observadores capaces de visualizar el estado de la partida sin participar activamente en ella.

El sistema implementa diversos mecanismos de sincronización para garantizar que todos los clientes mantengan una representación consistente del estado del juego, incluso cuando múltiples eventos ocurren simultáneamente.

2. Arquitectura General del Sistema

La arquitectura implementada sigue el modelo Cliente-Servidor.

2.1. Servidor Central

El servidor constituye el núcleo lógico del sistema. Todas las decisiones importantes del juego son procesadas en esta capa.

Sus responsabilidades principales son:

- Gestionar conexiones de jugadores y espectadores.
- Mantener el estado global de la partida.
- Validar movimientos enviados por los clientes.
- Controlar la aparición y movimiento de enemigos.
- Administrar vidas, puntajes y rondas.
- Sincronizar el estado del juego entre todos los clientes.
- Notificar cambios mediante el patrón Observer.

2.2. Cliente Jugador

El cliente jugador representa la interfaz visual principal.

Sus responsabilidades son:

- Mostrar los elementos gráficos del juego.
- Capturar las entradas del usuario.
- Enviar comandos al servidor.
- Recibir actualizaciones del estado global.
- Renderizar enemigos, jugador, bunkers y proyectiles.

2.3. Cliente Observador

El cliente observador permite visualizar una partida sin participar activamente en ella.

Este cliente recibe exactamente la misma información que los jugadores activos, pero no posee permisos para enviar comandos que modifiquen el estado del juego.

2.4. Controlador Raspberry Pi Pico

La Raspberry Pi Pico actúa como un dispositivo físico externo que permitirá controlar determinadas acciones del jugador mediante botones físicos conectados a pines GPIO.

La comunicación entre la Pico y el cliente se realizará mediante protocolos UART o I2C.

3. Descripción de las Estructuras de Datos

3.1. Estructura Canon

Representa al jugador dentro del cliente desarrollado en lenguaje C.

```
typedef struct {  
float x;  
float y;  
int vidas;  
int puntaje;  
} Canon;
```

Descripción de campos

- x: posición horizontal.
- y: posición vertical.
- vidas: cantidad de vidas restantes.
- puntaje: puntuación acumulada.

3.2. Estructura Bala

Representa un proyectil disparado por el jugador.

```
typedef struct {  
float x;  
float y;  
bool activa;  
} Bala;
```

3.3. Estructura Alien

Representa a un enemigo dentro de la partida.

```
typedef struct {  
float x;  
float y;  
bool vivo;  
} Alien;
```

3.4. Clase Player

Representa un jugador conectado al servidor.
Información almacenada:

- Identificador único.
- Posición horizontal.
- Cantidad de vidas.
- Puntaje acumulado.

3.5. Clase Bunker

Representa una estructura defensiva que puede recibir daño.
Información almacenada:

- Identificador.
- Posición.
- Vida restante.

3.6. Clase UFO

Representa el OVNI especial que aparece periódicamente durante la partida.
Información almacenada:

- Identificador.
- Dirección de movimiento.
- Posición.
- Puntaje otorgado.

4. Descripción de los Algoritmos Principales

4.1. Algoritmo de Conexión al Servidor

Este algoritmo permite establecer una conexión TCP entre el cliente y el servidor.

Pasos:

1. Inicializar Winsock.
2. Crear el socket TCP.
3. Configurar dirección IP y puerto.
4. Ejecutar la llamada connect().
5. Verificar errores.
6. Mantener la conexión abierta para intercambio de mensajes.

4.2. Algoritmo de Asignación de Clientes

Cuando un cliente se conecta:

1. El servidor consulta la cantidad de jugadores activos.
2. Si existen menos de dos jugadores:
 - Se crea un nuevo jugador.
 - Se asigna un identificador.
3. Si ya existen dos jugadores:
 - El cliente se convierte en espectador.

4.3. Algoritmo de Movimiento del Jugador

Pasos:

1. El usuario presiona una tecla.
2. El cliente genera un comando MOVE.
3. El comando es enviado al servidor.
4. El servidor valida la acción.
5. Se actualiza la posición.
6. Se notifica a todos los clientes.

4.4. Algoritmo de Eliminación de Aliens

1. El cliente detecta una colisión.
2. Envía el mensaje ALIEN KILLED.
3. El servidor localiza el alien correspondiente.
4. Verifica que continúe vivo.
5. Lo marca como eliminado.
6. Suma puntos al jugador.
7. Actualiza el estado global.
8. Notifica a todos los clientes.

4.5. Algoritmo de Reinicio de Ronda

Cuando todos los aliens han sido destruidos:

1. Se incrementa la velocidad del juego.
2. Se restauran los bunkers.
3. Se generan nuevos enemigos.
4. Se reinician posiciones.
5. Se notifica a los clientes.

4.6. Algoritmo de Movimiento de Aliens

Los aliens se desplazan horizontalmente de forma sincronizada.

1. Se calcula la nueva posición.
2. Se verifica si algún alien alcanzó un borde.
3. Si existe colisión con un borde:
 - Se invierte la dirección.
 - Todos los aliens descienden una fila.
4. Si no existe colisión:
 - Continúan avanzando horizontalmente.

4.7. Algoritmo de Aparición del OVNI

1. El servidor mantiene un contador de ciclos.
2. Cuando se alcanza un valor aleatorio:
 - Se crea un OVNI.
 - Se asigna dirección.
 - Se asigna valor de puntos.
3. El OVNI comienza a desplazarse.

5. Protocolo de Comunicación

La comunicación utiliza sockets TCP.

Los comandos enviados por los clientes siguen una estructura textual.

5.1. Comandos de Entrada

Comando	Descripción
MOVE LEFT	Mover jugador a la izquierda
MOVE RIGHT	Mover jugador a la derecha
SHOOT	Disparar proyectil
ALIEN KILLED id	Reportar alien eliminado
PLAYER HIT	Reportar daño recibido
BUNKER HIT id	Reportar daño a bunker
UFO KILLED	Reportar destrucción de OVNI
ALIENS REACHED	Reportar llegada de enemigos a la base

5.2. Mensajes del Servidor

Mensaje	Descripción
PLAYER _I D	Identificador asignado
SPECTATOR	Cliente observador
STATE	Estado completo del juego
GAME _O VER	Fin de partida

6. Patrones de Diseño Utilizados

6.1. Patrón Observer

Este patrón permite desacoplar el estado interno del juego de los clientes conectados. Cuando ocurre una modificación:

1. Se actualiza el GameState.
2. Se ejecuta notifyObservers().

3. Todos los observadores reciben el nuevo estado.

Ventajas:

- Sincronización automática.
- Bajo acoplamiento.
- Facilidad para agregar nuevos clientes.

6.2. Patrón Singleton

El proyecto utiliza un Singleton para administrar el ThreadPool encargado de ejecutar tareas concurrentes.

Beneficios:

- Única instancia global.
- Control centralizado.
- Menor consumo de recursos.

7. Problemas Encontrados

Durante el desarrollo se identificaron los siguientes problemas:

- Sincronización entre múltiples clientes.
- Manejo concurrente de conexiones.
- Actualización simultánea del estado global.
- Integración entre Java y C.
- Diseño del protocolo de comunicación.
- Gestión de desconexiones inesperadas.
- Coordinación entre cliente gráfico y servidor lógico.

8. Problemas sin Solución

- Reconexión automática de clientes.
- Persistencia de partidas.
- Recuperación de estado después de una caída del servidor.
- Comunicación cifrada.
- Balanceo de carga para múltiples partidas simultáneas.

9. Plan de Actividades

Actividad	Responsable	Fecha	Tiempo Estimado
Reunión para revisar y repartir responsabilidades	Todos	16/05/2026	1 Hora
Se crea el repositorio y README del proyecto en GitHub	Meibel Mora	14/05/2026	1 Hora
Se crea la carpeta modelos con la clase Player.java y los atributos id, vidas, puntaje y posición	Meibel Mora	20/05/2026	2 Horas
Se creó la clase Bunker.java que representa los cuatro escudos de protección del jugador	Meibel Mora	23/05/2026	2 Horas
Se creó la clase GameState.java que centraliza el estado del juego: jugadores, aliens y bunkers	Meibel Mora	25/05/2026	5 Horas
Se creó la clase TestClient.java para probar la conexión con el servidor	Meibel Mora	26/05/2026	2 Horas
Se implementó el comando PLAYER.HIT en ClientHandler y GameState para reducir la vida del jugador cuando es impactado por un alien. Además se implementó la condición de Game Over cuando el jugador pierde todas sus vidas	Meibel Mora	26/05/2026	1 Hora 30 min
Se agregaron los métodos allAliensDead() y resetRound() para detectar cuando todos los aliens mueren y reiniciar la ronda con mayor velocidad	Meibel Mora	29/05/2026	1 Hora
Se agregaron los métodos getAlienSpeed() y setBunkersHealth() para modificar en tiempo real la velocidad de los aliens y el estado de los bunkers	Meibel Mora	30/05/2026	1 Hora 30 min
Se creó la clase UFO.java para representar el platillo volador con dirección y puntos. Se agregaron los métodos de creación y destrucción del UFO en GameState y ClientHandler	Meibel Mora	31/05/2026	2 Horas 30 min
Se implementó el patrón Observer con las clases GameObserver.java y GameSubject.java en el paquete Patrones	Meibel Mora	01/06/2026	5 Horas
Se implementó el patrón Abstract Factory con las clases GameElementFactory.java, ClassicGameElementFactory.java y HardGameElementFactory.java	Meibel Mora	06/06/2026	8 Horas
Se crearon las subclases de aliens: CalamarAlien.java, CangrejoAlien.java y PulpoAlien.java	Meibel Mora	07/06/2026	2 Horas 30 min
Se implementó el patrón Singleton con ThreadExecutorSingleton.java para gestionar el pool de hilos del servidor	Meibel Mora	08/06/2026	3 Horas

Actividad	Responsable	Fecha	Tiempo Estimado
Se creó AdminHandler.java para manejar las conexiones del administrador en el puerto 5001 y se actualizó Main.java con un segundo ServerSocket	Meibel Mora	13/06/2026	2 Horas
Se refactorizó GameState dividiéndolo en AlienManager.java, UFOManager.java, GameLoop.java y GameState.java para separar responsabilidades	Meibel Mora	13/06/2026	1 Hora
Se crea el diagrama de clases del servidor	Meibel Mora	14/06/2026	2 Horas 30 min
Investigación de alternativas para el control físico del juego mediante microcontroladores	Yerik Jaslin Castro	15/05/2026	2 Horas
Búsqueda y selección de componentes electrónicos necesarios (Raspberry Pi Pico, botones, protoboard y cables)	Yerik Jaslin Castro	16/05/2026	2 Horas
Compra y adquisición de componentes para el prototipo	Yerik Jaslin Castro	17/05/2026	1 Hora
Diseño preliminar del circuito de control para el juego	Yerik Jaslin Castro	18/05/2026	2 Horas
Ensamblaje físico del circuito en protoboard	Yerik Jaslin Castro	19/05/2026	3 Horas
Configuración del entorno de desarrollo para Raspberry Pi Pico	Yerik Jaslin Castro	20/05/2026	2 Horas
Desarrollo del programa embebido para la lectura de botones	Yerik Jaslin Castro	21/05/2026	4 Horas
Implementación de comunicación serial USB entre la Pico y el cliente del juego	Yerik Jaslin Castro	22/05/2026	4 Horas
Pruebas de transmisión de comandos mediante PuTTY	Yerik Jaslin Castro	23/05/2026	2 Horas
Desarrollo del módulo pico_input.c para lectura del puerto serial	Yerik Jaslin Castro	24/05/2026	4 Horas
Integración de los comandos LEFT, RIGHT y SHOOT con el cliente Space Invaders	Yerik Jaslin Castro	25/05/2026	4 Horas
Pruebas de movimiento y disparo mediante botones físicos	Yerik Jaslin Castro	26/05/2026	3 Horas
Corrección de errores de comunicación serial y sincronización	Yerik Jaslin Castro	27/05/2026	3 Horas
Optimización de sensibilidad y tiempos de respuesta de los botones	Yerik Jaslin Castro	28/05/2026	2 Horas
Documentación técnica de la integración Raspberry Pi Pico – Cliente C	Yerik Jaslin Castro	29/05/2026	3 Horas
Elaboración de manual de usuario y capturas de funcionamiento	Yerik Jaslin Castro	08/06/2026	2 Horas

Actividad	Responsable	Fecha	Tiempo Estimado
Pruebas finales y validación de la funcionalidad completa	Yerik Jaslin Castro	09/06/2026	2 Horas
Implementación de pantalla de fin de juego y menú principal	Joseph Solorzano Jackson	06/04/2026	2 horas
Desarrollo de detección de victoria	Meibel Mora Brenes	06/04/2026	1 hora
Implementación del panel de fichas disponibles (modo colocar)	Joseph Solorzano Jackson	06/04/2026	2 horas
Desarrollo de detección de fin del juego	Meibel Mora Brenes	06/04/2026	1.5 horas
Definición del estado anidado y funciones de acceso	Joseph Solorzano Jackson	07/04/2026	1.5 horas
Implementación del estado inicial y funciones auxiliares del juego	Joseph Solorzano Jackson	07/04/2026	1 hora
Implementación de generación de ficha aleatoria	Meibel Mora Brenes	08/04/2026	2 horas
Implementación de cálculo de puntajes por movimiento	Meibel Mora Brenes	08/04/2026	2 horas
Desarrollo función general del juego	Meibel Mora Brenes	08/04/2026	2 horas
Implementación de render-juego y render-colocar con tablero escalado	Joseph Solorzano Jackson	08/04/2026	2 horas
Implementación de dibujar-celda-cursor y dibujar-tablero-cursor	Joseph Solorzano Jackson	08/04/2026	2 horas
Implementación de funciones de movimiento del cursor	Joseph Solorzano Jackson	08/04/2026	1 hora
Implementación de manejar-tecla-elegir-celda	Joseph Solorzano Jackson	08/04/2026	1.5 horas
Implementación de manejar-tecla-colocar con navegación de fichas	Joseph Solorzano Jackson	08/04/2026	1.5 horas
Corrección de error: funciones auxiliares de navegación no definidas	Joseph Solorzano Jackson	09/04/2026	30 min
Implementación de manejar-tecla-menu y manejar-tecla-juego	Joseph Solorzano Jackson	09/04/2026	1 hora
Implementación del despachador render y manejar-tecla + big-bang	Joseph Solorzano Jackson	09/04/2026	1 hora
Implementación de configuración de filas y columnas	Joseph Solorzano Jackson	09/04/2026	2 horas
Unión del código y documentación en un solo .zip	Todos	09/04/2026	1 hora
Investigación de la biblioteca Raylib y planificación de módulos del cliente	Joseph Solórzano	15/05/2026	2 Horas

Actividad	Responsable	Fecha	Tiempo Estimado
Configuración del entorno de desarrollo en Windows (MinGW, Raylib y WinSock2)	Joseph Solórzano	16/05/2026	2 Horas
Definición de constantes globales en constants.h (pantalla, red, sprites y colores)	Joseph Solórzano	17/05/2026	1 Hora
Diseño e implementación de las estructuras de datos en game_state.h (Alien, Bunker, UFO, Jugador, Bala y BalaEnemiga)	Joseph Solórzano	18/05/2026	1 Hora 30 min
Implementación del módulo de red TCP con WinSock2 (network.c y network.h): conexión, envío y recepción no bloqueante	Joseph Solórzano	19/05/2026	3 Horas
Desarrollo del buffer acumulador de mensajes TCP y función procesarMensaje() en net_handler.c	Joseph Solórzano	20/05/2026	3 Horas
Implementación del parseo del protocolo STATE: reconstrucción de aliens, bunkers, UFO, jugadores y SPEED	Joseph Solórzano	21/05/2026	3 Horas
Implementación del menú de selección de rol (P1/P2/Espectador) y lógica de estados en menu.c	Joseph Solórzano	22/05/2026	1 Hora 30 min
Desarrollo del algoritmo de movimiento predictivo de aliens (actualizarAliensLocal) con rebote y descenso	Joseph Solórzano	23/05/2026	2 Horas
Implementación del movimiento del UFO y detección de salida del área de juego	Joseph Solórzano	24/05/2026	1 Hora 30 min
Implementación del disparo aleatorio de aliens (actualizarDisparoAliens) con temporizador por frames	Joseph Solórzano	25/05/2026	3 Horas
Implementación de detección de colisiones AABB: bala del jugador contra aliens, UFO y bunkers	Joseph Solórzano	26/05/2026	2 Horas
Implementación de colisiones de balas enemigas contra bunkers y el jugador local con envío de PLAYER_HIT	Joseph Solórzano	27/05/2026	1 Hora 30 min
Desarrollo del renderer completo en renderer.c: escena, menú, game over y modo sin conexión	Joseph Solórzano	28/05/2026	2 Horas
Integración del game loop principal en main.c: lectura de red, input, update y draw por frame	Joseph Solórzano	29/05/2026	2 Horas
Detección de nueva ronda por cambio de SPEED y reinicio de formación y balas sin perder jugadores ni bunkers	Joseph Solórzano	30/05/2026	1 Hora 30 min
Pruebas de integración cliente-servidor: validación de mensajes ALIEN_KILLED, BUNKER_HIT y UFO_KILLED	Joseph Solórzano	31/05/2026	2 Horas
Corrección de errores de sincronización de posición del jugador remoto y espectador	Joseph Solórzano	01/06/2026	3 Horas

10. Conclusiones

La implementación del proyecto permitió aplicar de manera práctica conceptos fundamentales de programación orientada a objetos, sistemas distribuidos, comunicación mediante sockets y diseño de software basado en patrones.

La arquitectura cliente-servidor facilitó la centralización de la lógica del juego y permitió mantener sincronizados múltiples clientes conectados simultáneamente.

La utilización del patrón Observer simplificó significativamente la propagación de cambios hacia los clientes, mientras que el patrón Singleton permitió una administración eficiente de los recursos asociados a la concurrencia.

Finalmente, la incorporación de un cliente observador y la futura integración de hardware mediante Raspberry Pi Pico amplían las capacidades del sistema y demuestran la flexibilidad de la arquitectura desarrollada.

11. Recomendaciones

- Implementar mecanismos de reconexión automática.
- Agregar persistencia de datos.
- Incorporar autenticación de usuarios.
- Implementar cifrado de comunicaciones.
- Mejorar el protocolo de sincronización.
- Incorporar nuevas mecánicas de juego.
- Realizar pruebas de carga con múltiples clientes.
- Optimizar el consumo de ancho de banda.
- Extender la integración con Raspberry Pi Pico.
- Documentar futuras versiones siguiendo el mismo estándar técnico.